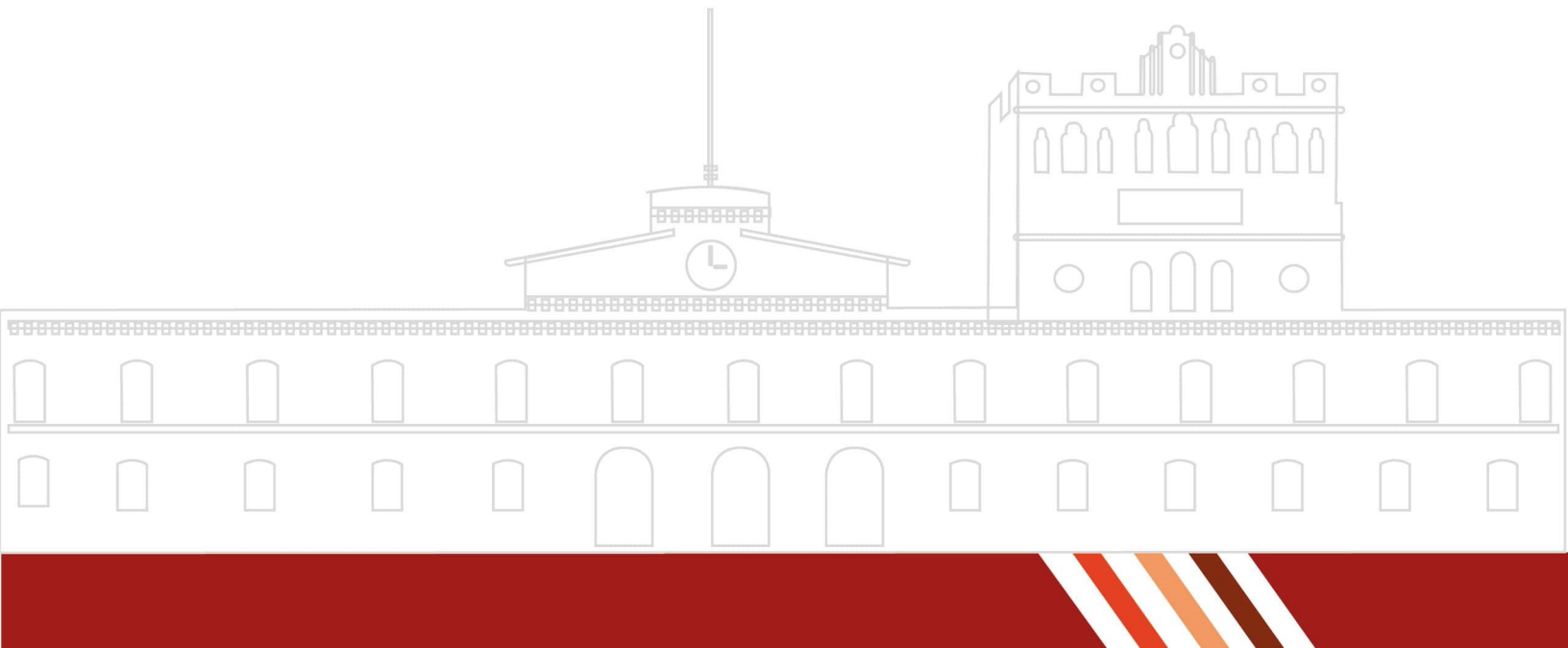


REPORTE DE PRÁCTICA NO. 1

ANALIZADOR LÉXICO

ALUMNO:
SAMUEL ORTIZ SALINAS

Dr. Eduardo Cornejo-Velázquez



1. Introducción

Esta práctica consiste en el diseño e implementación de un **analizador léxico**, es decir, la primera fase de un compilador cuya función es recorrer una cadena de entrada (código fuente) y reconocer los patrones (lexemas) que la componen, clasificándolos en categorías de tokens: números enteros, palabras en minúsculas, palabras en mayúsculas, identificadores y símbolos.

El proceso sistemático y metodológico para construir un analizador léxico es fundamental porque constituye la base sobre la cual se apoyan las siguientes fases de un compilador (análisis sintáctico y semántico). Un reconocimiento incorrecto de los lexemas provoca errores en cascada en todo el proceso de compilación.

En esta práctica se implementaron dos enfoques distintos para resolver el mismo problema: uno basado en **estructuras de datos** (un autómata finito determinista programado a mano) y otro basado en **expresiones regulares**, integrados en una interfaz gráfica desarrollada con Qt.

2. Marco teórico

Análisis léxico

El análisis léxico es la etapa de un compilador encargada de convertir una secuencia de caracteres en una secuencia de tokens, agrupando los caracteres en unidades con significado (lexemas) y asignándoles una categoría gramatical [1].

Autómatas finitos deterministas

Un autómata finito determinista (AFD) es un modelo computacional formado por un conjunto finito de estados, un alfabeto de entrada y una función de transición que, para cada estado y símbolo de entrada, determina un único estado siguiente. Los AFD son la herramienta teórica estándar para reconocer lenguajes regulares y, por tanto, para construir analizadores léxicos [1].

Expresiones regulares

Las expresiones regulares son una notación formal que describe patrones dentro de cadenas de texto y que, al ser equivalentes en poder expresivo a los autómatas finitos, permiten especificar de forma compacta los lexemas válidos de cada categoría de token (dígitos, letras, identificadores, símbolos) [1].

Alfabeto (Σ)

El alfabeto Σ de un lenguaje formal es el conjunto finito de símbolos válidos a partir de los cuales se construyen las cadenas del lenguaje. En el caso de un analizador léxico, Σ está compuesto típicamente por dígitos, letras y símbolos especiales del lenguaje de programación a reconocer [1].

3. Herramientas empleadas

1. **C++17**. Lenguaje de programación utilizado para implementar tanto el autómata manual como el analizador basado en expresiones regulares mediante la biblioteca estándar `<regex>`.
2. **Qt (Widgets)**. Framework utilizado para construir la interfaz gráfica de escritorio que permite ingresar el texto a analizar y visualizar los tokens reconocidos y su conteo por categoría.
3. **CMake**. Sistema de generación de proyectos empleado para compilar la aplicación de forma multi-plataforma.

4. Desarrollo

Análisis de requisitos

El sistema debe permitir al usuario escribir libremente una cadena de texto en una caja de texto y, al presionar un botón “Analizar”, mostrar:

- La lista de lexemas reconocidos junto con su tipo de token.
- El conteo total de tokens por cada categoría: enteros, palabras en minúsculas, palabras en mayúsculas, identificadores y símbolos.

Los tipos de token a reconocer son: número entero (con signo opcional), palabra en minúsculas, palabra en mayúsculas, identificador (letras, dígitos y guion bajo) y símbolo, incluyendo símbolos compuestos como ==, !=, <=, ++, entre otros.

Definición del alfabeto y expresiones regulares

En la Tabla 1 se presenta el alfabeto Σ definido para el lenguaje reconocido y las expresiones regulares asociadas a cada categoría de token.

Table 1: Expresiones regulares por categoría de token.

Categoría	Expresión regular
Número entero	<code>[+-]?[0-9]+</code>
Palabra minúscula	<code>[a-z]+</code>
Palabra mayúscula	<code>[A-Z]+</code>
Identificador	<code>[A-Za-z_][A-Za-z0-9_]*</code>
Símbolo compuesto	<code>== != <= >= ++ -- += -= *= /= && << >></code>
Símbolo	<code>[^A-Za-z0-9_ \s]</code>

Autómata finito (enfoque de estructuras de datos)

El primer enfoque implementa un autómata finito determinista sin usar expresiones regulares. La cadena de entrada se copia a un arreglo de caracteres del tamaño del texto y se recorre posición por posición: mientras el carácter actual pertenezca a la misma clase (dígito, letra o guion bajo) se sigue acumulando en el lexema actual.

Al encontrar un espacio, un salto de línea o un cambio de clase no compatible, se cierra el lexema y se clasifica según su contenido.

Este enfoque contempla además casos especiales de dos caracteres de anticipación (*lookahead*) para reconocer números con signo (-5, +5) y símbolos compuestos (==, !=, <=, >=, ++, --, +=, -=, *=, /=, &&, ||, <<, >>) como un único token, en lugar de dividirlos en símbolos individuales.

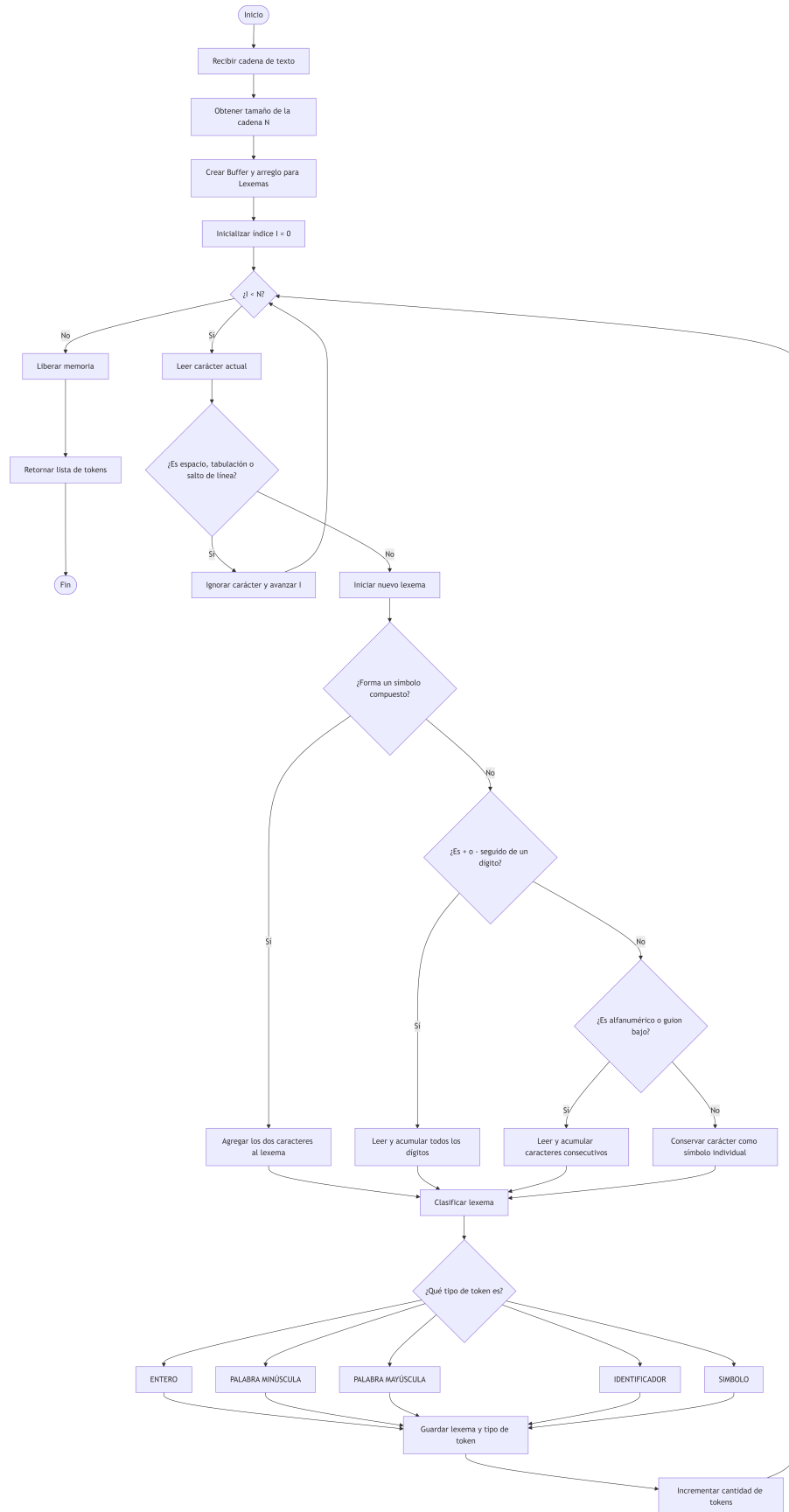


Figure 1: Diagrama de flujo del proceso de análisis léxico basado en estructuras de datos.

Analizador basado en expresiones regulares

El segundo enfoque utiliza la biblioteca `<regex>` de C++. Se define una expresión regular “tokenizadora” (Tabla 1) que recorre la cadena completa extrayendo cada lexema.

Posteriormente, cada lexema extraído se compara contra las expresiones regulares individuales de cada categoría, ancladas con `^` y `$`, para determinar su tipo.

Interfaz gráfica

La interfaz, desarrollada en Qt, integra ambos enfoques en pestañas independientes (ver Figura 2): una pestaña para el autómata de estructuras de datos, una pestaña con la descripción formal del alfabeto y las expresiones regulares, y una pestaña para el analizador basado en regex con su respectiva tabla de conteo por categoría.

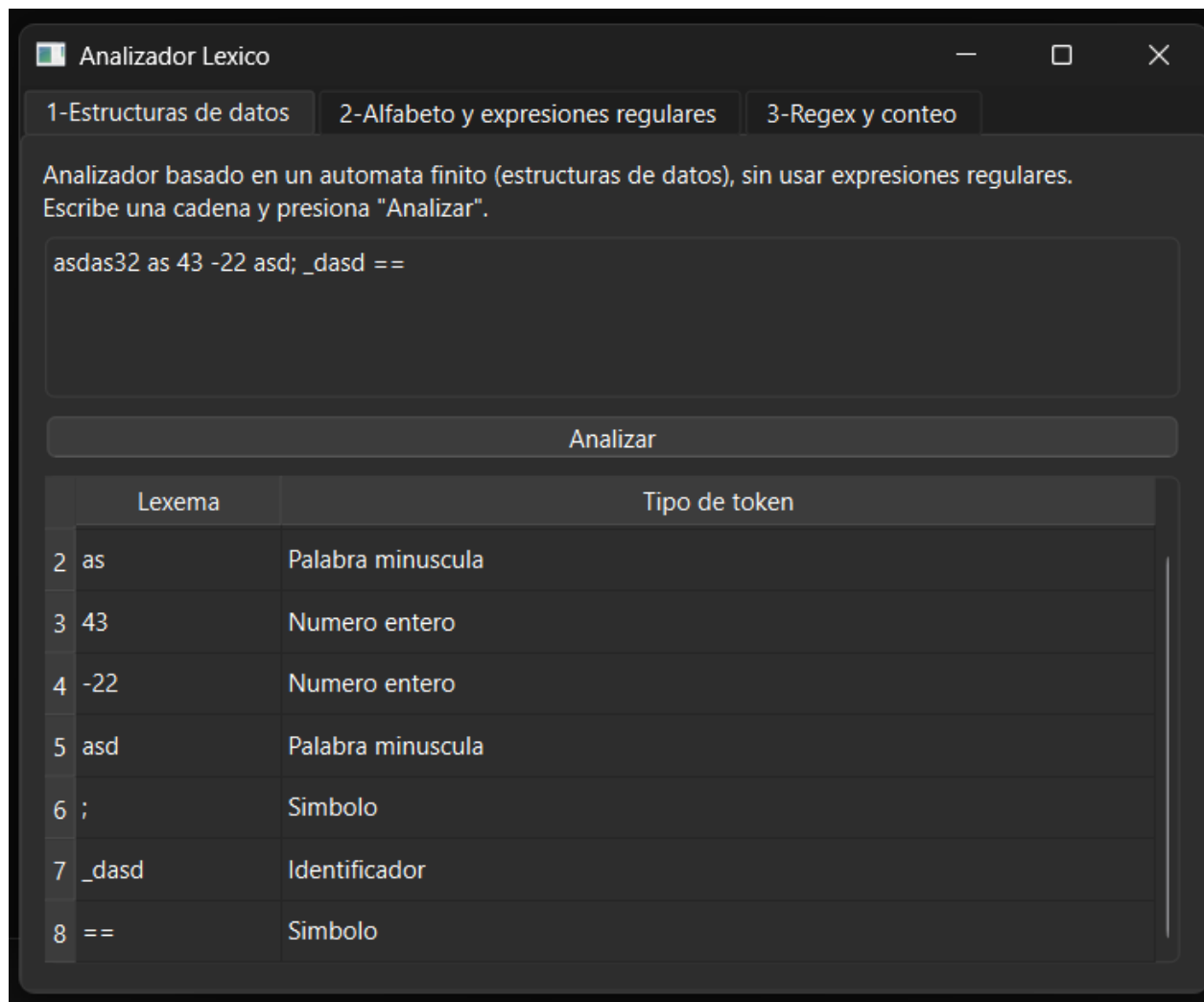


Figure 2: Interfaz gráfica del analizador léxico.

Fragmento de código representativo

En el Listado 1 se presenta la función de clasificación de lexemas utilizada por el autómata de estructuras de datos.

Listing 1: Clasificación de un lexema según sus caracteres.

```

1
2 TipoToken AutomataLexico::clasificar(const string &Lexema)
3 {
4     bool TieneDigito = false, TieneMinuscula = false;
5     bool TieneMayuscula = false, TieneGuion = false;
6     bool EsValido = true;
7
8     for (char C : Lexema)
9     {
10         if (isdigit((unsigned char)C))
11             TieneDigito = true;
12
13         else if (islower((unsigned char)C))
14             TieneMinuscula = true;
15
16         else if (isupper((unsigned char)C))
17             TieneMayuscula = true;
18
19         else if (C == '_' || C == '-')
20             TieneGuion = true;
21
22         else
23             EsValido = false;
24     }
25
26     if (!EsValido)
27         return TipoToken::SIMBOLO;
28
29     if (TieneGuion ||
30         (TieneDigito && (TieneMinuscula || TieneMayuscula)) ||
31         (TieneMinuscula && TieneMayuscula))
32     {
33         return TipoToken::IDENTIFICADOR;
34     }
35
36     if (TieneDigito)
37         return TipoToken::ENTERO;
38
39     if (TieneMinuscula)
40         return TipoToken::PALABRA_MINUSCULA;
41
42     if (TieneMayuscula)
43         return TipoToken::PALABRA_MAYUSCULA;
44
45     return TipoToken::SIMBOLO;
46 }

```


5. Conclusiones

Al implementar el mismo problema con dos enfoques distintos, autómata manual y expresiones regulares, se reforzó la comprensión de la equivalencia teórica entre autómatas finitos y expresiones regulares, así como sus diferencias prácticas.

El autómata manual ofrece control total sobre el proceso de reconocimiento, resultando útil para casos especiales como símbolos compuestos o números con signo. Por otro lado, el enfoque con expresiones regulares resulta mucho más compacto y declarativo, aunque ofrece menor control fino sobre el proceso carácter por carácter.

Integrar ambos enfoques en una misma interfaz gráfica permitió comparar directamente sus resultados sobre las mismas cadenas de entrada.

Referencias Bibliográficas

References

- [1] Carranza Sahagún, D. U., y Nolaso Salcedo, M. del C. (2024). *Análisis léxico*. En D. U. Carranza Sahagún (Coord.), *Compiladores: fases de análisis*. Editorial Transdigital. ISBN: 978-607-26754-0-7.